

Complemento Práctico: Control y Generalización

Si la Actividad 1 fue sobre "Qué es la Herencia", esta actividad es sobre "**Cómo gestionarla sin romper nada**". Analizaremos el modificador `protected` (el término medio entre lo privado y lo público), resolveremos el error de compilación más común al heredar ("Constructor not found") entendiendo el rol de `super`, y aprenderemos a mirar nuestros objetos con "lentes genéricos" a través del **Upcasting**, la base de las colecciones polimórficas.

En esta actividad vamos a profundizar en tres pilares fundamentales que todo programador debe dominar para trabajar efectivamente con herencia en Java. Estos conceptos son las herramientas que les permitirán crear jerarquías de clases robustas, mantener la integridad de sus objetos y escribir código más flexible y reutilizable.



Contenido

Visibilidad Protected

La diferencia sutil entre `protected` y `package-private`. El término medio perfecto para la herencia.

Constructores

Por qué no se heredan y la obligación de llamar a `super()` para inicializar correctamente.

Upcasting

Guardar un objeto específico en una variable genérica. La base del polimorfismo.

Instanceof

La pregunta de identidad: ¿qué tipo de objeto es realmente en memoria?

El Modificador Protected: "Solo para la Familia"

En el video aclaramos la confusión sobre los niveles de acceso. Sabemos que `private` es "solo yo" y `public` es "todos". El modificador `protected` (# en UML) es un punto medio diseñado específicamente para la herencia.

La Regla de Acceso

Si un atributo es `protected`:

1. Las **Subclases** (Hijos) pueden verlo directamente, **aunque estén en otro paquete**
2. Las clases del **Mismo Paquete** también pueden verlo (funciona como default)



❏ **Recomendación:** Úsenlo con discreción. Abrir un atributo a `protected` rompe un poco el encapsulamiento. Sigán usando atributos `private` + Getters `protected` o `public` si quieren mantener el control estricto.

Ejemplo Práctico de Protected

Veamos cómo funciona protected en la práctica con un ejemplo concreto que ilustra ambos escenarios de acceso:

Clase Padre (paquete modelo)

```
package modelo;

public class Padre {
    protected String secretoFamiliar
    = "Receta de la Abuela";
}
```

Clase Hijo (otro paquete)

```
package otro_paquete;
import modelo.Padre;

public class Hijo extends Padre {
    public void revelarSecreto() {
        // Al ser protected, el Hijo
        PUEDE acceder directo,
        // aunque esté en otro
        paquete.
        System.out.println("El secreto
        es: " + this.secretoFamiliar);
    }
}
```

¿Cuándo usarlo?

El modificador protected es útil cuando queremos permitir que las subclases accedan a ciertos miembros sin exponerlos completamente al público. Es especialmente común en frameworks y bibliotecas donde se espera que los usuarios extiendan clases base. Sin embargo, recuerden que cada vez que usan protected están creando una dependencia entre la clase padre y sus hijos, por lo que debe usarse conscientemente.

El Problema de los Constructores

¿Por qué Java es tan estricto con el "orden de nacimiento" en la herencia? Para comprender el famoso "Constructor not found" y el rol vital de `super()`, primero debemos entender cómo se organizan los objetos en la memoria. Cuando creamos una instancia de una subclase (por ejemplo, un objeto de la clase `Hijo`), Java construye una estructura en capas, similar a una cebolla.

En el núcleo de esta "cebolla" se encuentran los atributos y componentes de la clase `Padre`. Alrededor de esta base, o "encima", se ubican los atributos específicos de la clase `Hijo`. Esto refleja la naturaleza de la herencia: un hijo *es un* padre con características adicionales.

La Regla de Oro de la Inicialización: Java impone una condición fundamental: antes de que la parte correspondiente al `Hijo` pueda ser inicializada, la parte del `Padre` debe haber sido completamente construida y preparada. No podemos inicializar las características únicas del hijo si su base heredada no está lista.

Pensemos en una analogía arquitectónica: estamos construyendo una casa. La clase `Padre` representa los cimientos sólidos y las vigas maestras de la estructura. La clase `Hijo`, por otro lado, son las paredes, el techo, las ventanas y todos los detalles de decoración que hacen única a la casa.

El problema surge si intentamos pintar las paredes (inicializar el `Hijo`) antes de haber vertido y secado el cemento de los cimientos (inicializar el `Padre`). Esto es inviable. La llamada a `super()` dentro de un constructor de subclase es precisamente esa orden crucial: "construye primero los cimientos" o, en términos de Java, "ejecuta el constructor de la superclase para asegurar que la parte heredada esté correctamente inicializada".

Anatomía del Error Invisible

Aquí es donde la frustración alcanza su punto máximo para muchos desarrolladores: un error de compilación ocurre en una línea de código que ¡ni siquiera escribieron! Este fenómeno se centra en la desaparición misteriosa del constructor por defecto de la superclase y cómo puede desbaratar la inicialización de nuestras subclases.

El Comportamiento Normal: Funcionamiento "Invisible"

Si no defines ningún constructor explícitamente en tu clase Padre, Java te provee generosamente con uno por defecto (sin argumentos), invisible para nosotros: `public Padre() {}`. Gracias a esto, cualquier subclase Hijo puede hacer su llamada implícita `super()` en su propio constructor, y todo funciona a la perfección.

La Trampa: El Cambio que lo Rompe Todo

El problema surge cuando decides que tu clase Padre debe asegurar la inicialización de ciertos datos. En ese momento, escribes tu propio constructor con parámetros. Es crucial entender que, al hacer esto, **Java ELIMINA automáticamente el constructor vacío que había regalado**.

```
public class Padre {  
    // ¡ALERTA! Al escribir este constructor,  
    // Java ELIMINA el constructor vacío por defecto.  
    // Ahora, la ÚNICA forma de crear un Padre es  
    // proporcionando un String.  
    public Padre(String nombre) {  
        // ... inicialización con 'nombre'  
    }  
}
```

El Colapso Silencioso de la Subclase

Ahora, observa la clase `Hijo`. Parece completamente inocente y sin cambios. Su constructor sigue siendo simple y sin llamadas explícitas a `super()`.

```
public class Hijo extends Padre {  
    public Hijo() {  
        // ¡¡ERROR DE COMPILACIÓN AQUÍ!!  
        // Tú no ves nada, pero el compilador insertó: 'super();'  
        // Esto busca el constructor vacío de Padre, que ya no existe.  
    }  
}
```

El diagnóstico es claro: el compilador de Java intenta automáticamente llamar al constructor vacío de `Padre` (`super();`) desde el constructor de `Hijo`. Sin embargo, dado que definiste un constructor con parámetros en `Padre`, el constructor vacío predeterminado dejó de existir. Es como intentar llamar a un número de teléfono que fue dado de baja: simplemente no hay nadie al otro lado para responder.

La Solución Técnica: Tomando el Control con `super(...)`

Cuando un constructor por defecto de la superclase desaparece, el compilador nos exige tomar el control explícitamente. La solución no es complicada, pero requiere una intervención manual para indicar a la subclase cómo debe inicializar a su padre.

Como programadores, debemos comunicarle al hijo: "Tu padre ya no puede nacer sin un argumento; debes enviarle los datos que pide para su correcta inicialización". Esto se logra a través de la llamada explícita a `super(...)`.

- ❑ **Regla de Sintaxis Fundamental:** La llamada a `super(...)` debe ser **obligatoriamente la primera línea** de cualquier constructor de la subclase. No se puede ejecutar ninguna lógica del hijo antes de asegurar que la porción del padre esté completamente construida y lista.

```
public class Hijo extends Padre {  
  
    public Hijo() {  
        // SOLUCIÓN: Llamada explícita al constructor del Padre.  
        // Cumplimos el contrato del Padre enviándole el argumento requerido.  
        super("Nombre Genérico");  
  
        // Una vez que el padre ha sido inicializado, el hijo puede proceder  
        // con su propia lógica de inicialización.  
        System.out.println("Hijo creado exitosamente");  
    }  
}
```

En este ejemplo, al agregar `super("Nombre Genérico");`, estamos llamando al constructor de `Padre` que acepta un `String`. Esto resuelve el error de compilación porque garantizamos que la porción heredada del objeto `Hijo` se construya correctamente antes de que el propio `Hijo` intente inicializar sus partes. Es un paso crucial para mantener la coherencia y la integridad de la jerarquía de herencia en Java.

Caso de Uso Real: Sistema de E-Commerce

Para aterrizar la importancia del constructor de la superclase, dejemos de lado las analogías de "Padre/Hijo" y trasladémonos a un escenario tangible en ingeniería de software: un sistema de comercio electrónico. En este contexto, la correcta inicialización de objetos es crucial para la integridad de los datos y el funcionamiento del negocio.

Imaginemos que necesitamos gestionar diversos productos en nuestra plataforma. El diseño de nuestra aplicación establece dos tipos principales:

- Todo **Producto** debe tener un nombre y un precio. Es una regla de negocio fundamental que no permitimos productos anónimos o sin valor definido.
- Un **ProductoDigital** es una especialización de **Producto** que, además de sus características básicas, debe incluir un enlace de descarga.

Así se verían nuestras clases en código:

Superclase: **Producto**

```
public class Producto {  
    protected String nombre;  
    protected double precio;  
  
    // Constructor OBLIGATORIO.  
    // Al definir este, Java NO crea el constructor vacío por defecto.  
    public Producto(String nombre, double precio) {  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
}
```


Subclase: ProductoDigital

Aquí es donde aplicamos la solución de forma explícita. El constructor de `ProductoDigital` debe recibir todos los datos necesarios para inicializar tanto su parte heredada (como `Producto`) como sus propios atributos. Luego, delega la responsabilidad de la parte heredada a la superclase mediante `super(...)`.

```
public class ProductoDigital extends Producto {  
    private String linkDescarga;  
  
    // El constructor recibe 3 datos: 2 para el padre, 1 para sí mismo.  
    public ProductoDigital(String nombre, double precio, String link) {  
  
        // 1. PRIMERO: Cumplir con la superclase (Producto)  
        // La llamada a super(...) debe ser la primera línea.  
        super(nombre, precio);  
  
        // 2. SEGUNDO: Inicializar los atributos propios de ProductoDigital  
        this.linkDescarga = link;  
    }  
}
```

01

Delegación Primordial

La primera acción en el constructor de `ProductoDigital` es `super(nombre, precio)`. Esto asegura que la parte de "producto base" de nuestro objeto se inicialice correctamente, invocando el constructor de la superclase `Producto` con los valores requeridos.

02

Inicialización Propia

Solo después de que la parte `Producto` ha sido construida, `ProductoDigital` procede a inicializar su atributo específico, `linkDescarga`. Esta secuencia es vital.

03

Conclusión del Caso

Si omitiéramos la llamada a `super(nombre, precio)`, el compilador emitiría un error, ya que no sabría cómo inicializar la porción `Producto` del objeto. Gracias a `super`, los datos fluyen correctamente hacia arriba en la jerarquía, garantizando un objeto completo y válido.

Upcasting: Mirando el Panorama General

El **Upcasting** (Casteo hacia arriba) es tratar a un objeto específico como si fuera su padre genérico. Es seguro, automático y no pierde información. El objeto en memoria sigue siendo un Ferrari, solo restringe cómo lo vemos.

01	02	03
Creación del Objeto	Asignación Genérica	Restricción Visual
Instanciamos un objeto de la clase hija (específica)	Lo guardamos en una variable de tipo padre (genérica)	Solo podemos usar métodos del tipo padre

```
// Variable de tipo PADRE (Instrumento)
// Objeto de tipo HIJO (Guitarra)
Instrumento miInstrumento = new Guitarra();

// Esto es legal y automático en Java
```

Esto es legal y automático. Al guardar la guitarra en una caja de "Instrumento", **solo podemos usar los métodos de Instrumento**. Si Guitarra tiene el método `tocarSolo()`, no podremos usarlo desde la variable `miInstrumento`, porque Java la ve como un instrumento genérico.

Beneficios del Upcasting

¿Por qué es útil?

El upcasting es la base del polimorfismo y nos permite escribir código más flexible y reutilizable. Es fundamental para trabajar con colecciones heterogéneas.

Aplicaciones Prácticas

- Crear `ArrayList<Mascota>` que contenga perros, gatos y peces
- Escribir métodos que acepten tipos genéricos y trabajen con cualquier subtipo
- Implementar patrones de diseño como Strategy o Factory
- Simplificar el código eliminando duplicación

Colecciones Polimórficas

Podemos tener una lista de animales que contenga diferentes tipos específicos, todos tratados uniformemente.

Métodos Genéricos

Escribimos un método una vez que funciona con el tipo padre y automáticamente acepta todos los hijos.

Código Flexible

El upcasting permite que nuestro código sea más adaptable a cambios futuros y nuevas subclases.

Instanceof: La Pregunta de Identidad

Si recibimos un objeto genérico (ej: Mascota m), a veces necesitamos saber si en el fondo es un Perro o un Gato para poder usar sus métodos específicos (Downcasting). El operador instanceof devuelve true o false preguntando por el tipo real en memoria.

Ejemplo Práctico

```
public void jugar(Mascota m) {  
    // 'm' puede ser cualquier cosa.  
    Preguntamos:  
    if (m instanceof Perro) {  
        // Si es perro, es seguro  
        castear  
        // y usar métodos de perro  
        Perro p = (Perro) m;  
        p.ladrar();  
        System.out.println("Le tiro el  
hueso.");  
    } else if (m instanceof Gato) {  
        System.out.println("Es un  
gato, uso el láser.");  
    }  
}
```



❏ **Advertencia del Video:** No abusen de esto. Si tienen muchos `if (x instanceof ...)`, probablemente deberían estar usando **Polimorfismo** (que veremos en la Actividad 4).

El instanceof es una herramienta útil pero debe usarse con moderación. En la mayoría de los casos, un buen diseño orientado a objetos con polimorfismo elimina la necesidad de verificar tipos explícitamente. Piénsenlo como un recurso de último caso cuando realmente necesiten distinguir entre subtipos.

Resumen: Ajustando las Tuercas de la Herencia

Hemos completado un recorrido exhaustivo por los aspectos más finos de la herencia en Java. Estos conceptos son fundamentales para escribir código profesional y evitar los errores más comunes que enfrentan los programadores al trabajar con jerarquías de clases.

1. Seguridad con Protected

Usamos `protected` para compartir datos con hijos sin abrirlos al mundo entero. Es el equilibrio perfecto entre encapsulamiento y herencia.

2. Inicialización Correcta

Entendemos que el Hijo es responsable de llamar al constructor del Padre con `super(...)` si el padre no tiene constructor vacío.

3. Flexibilidad con Upcasting

Con el Upcasting, podemos crear listas genéricas y guardar adentro objetos de diferentes subtipos, tratándolos a todos por igual.

4. Verificación con Instanceof

Cuando sea absolutamente necesario, usamos `instanceof` para verificar el tipo real de un objeto y hacer downcasting seguro.

Con estos conceptos dominados, están listos para avanzar hacia el polimorfismo y aprovechar todo el poder de la programación orientada a objetos. En la próxima actividad veremos cómo el polimorfismo elimina la necesidad de muchos `instanceof` y hace nuestro código aún más elegante y mantenible. ¡Excelente trabajo!